School of Computer Science and Artificial Intelligence

Lab Assignment # 08

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task Description #1 (Password Strength Validator – Apply AI in Security Context)
• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

• Requirements:
o Password must have at least 8 characters.
o Must include uppercase, lowercase, digit, and special character.
o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1: • Password validation logic passing all AI-generated test cases.

Output Screenshot:

Explanation: This checks if a password is truly “strong” by making sure it has the right mix of characters and no spaces.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

• Requirements:

- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"

Expected Output #2: • Classification logic passing all assert tests.

Output Screenshot:

Explanation: This sorts numbers into positive, negative, or zero, while politely rejecting anything that isn’t a number.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

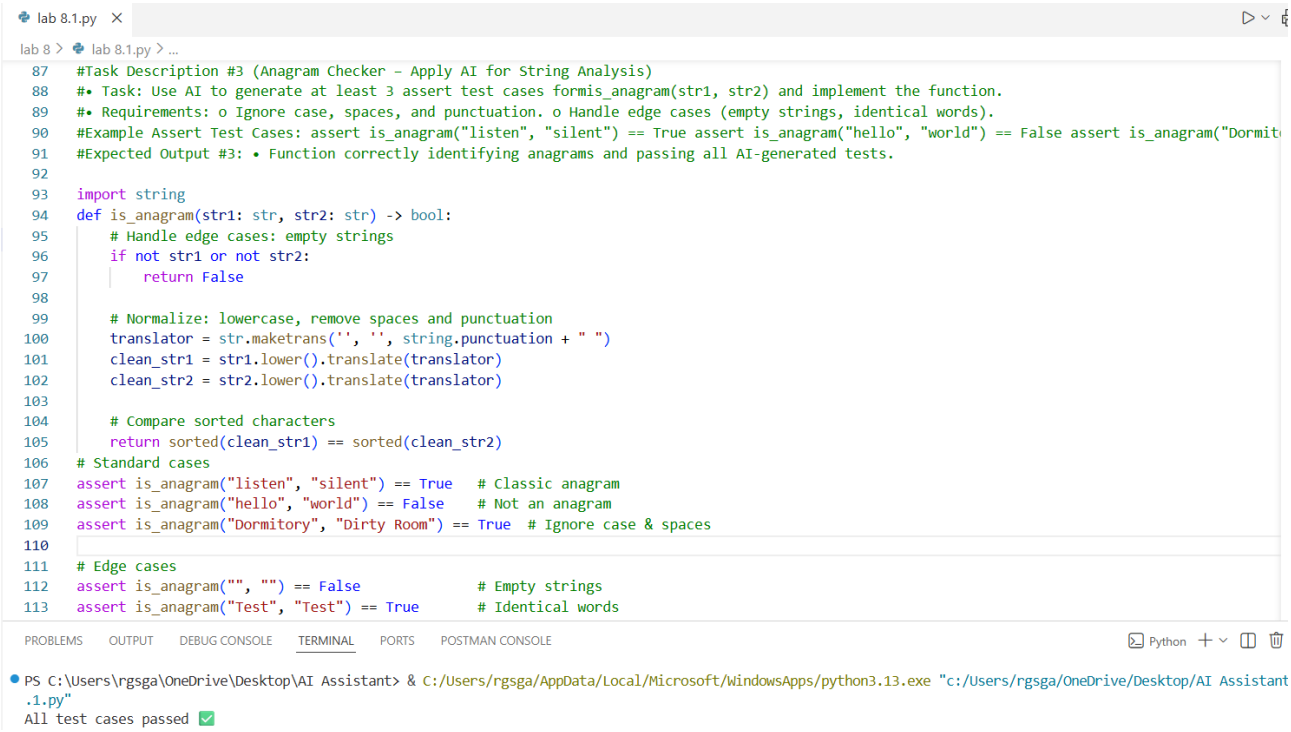
• Task: Use AI to generate at least 3 assert test cases formis_anagram(str1, str2) and implement the function.

• **Requirements:**
  o Ignore case, spaces, and punctuation.
  o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:
  assert is_anagram("listen", "silent") == True
  assert is_anagram("hello", "world") == False
  assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3: • Function correctly identifying anagrams and passing all AI-generated tests.

Output Screenshot:



Explanation: This spots whether two words or phrases are made of the same letters, ignoring case, spaces, and punctuation.

Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)

• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

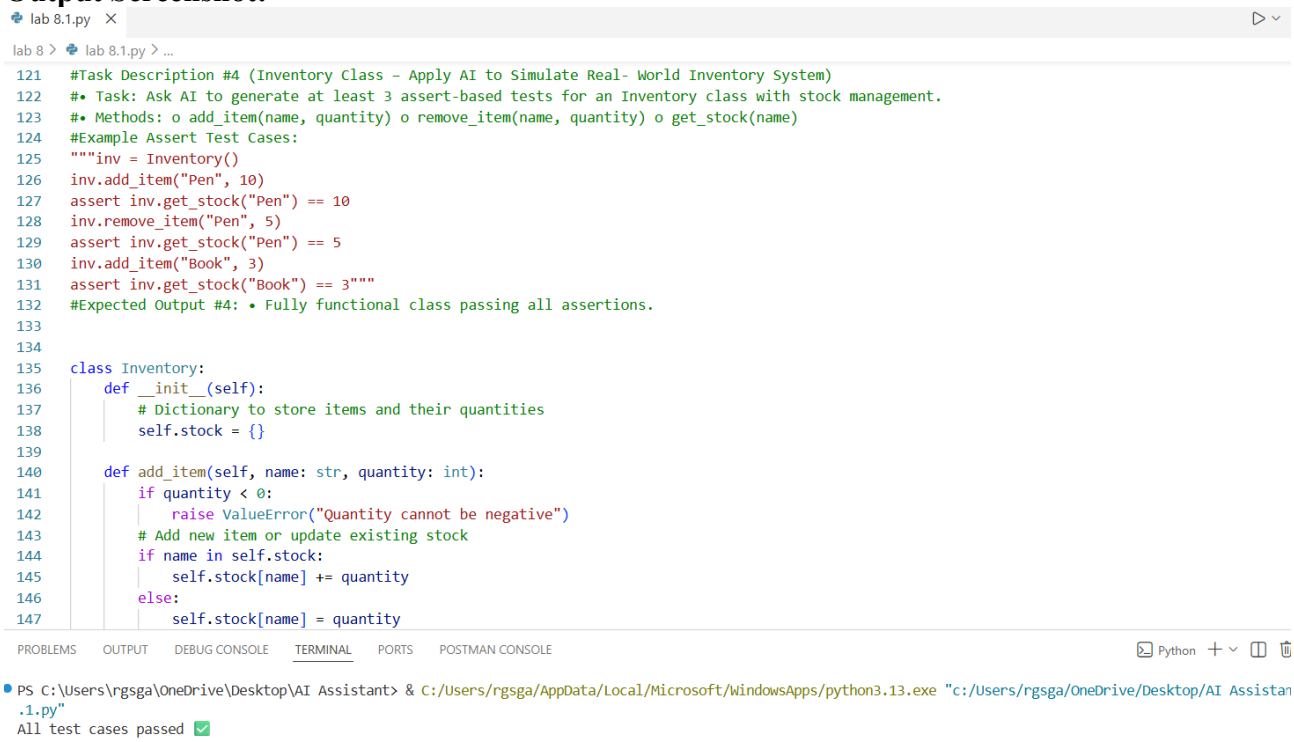
• **Methods:**
  o add_item(name, quantity)
  o remove_item(name, quantity)
  o get_stock(name)

Example Assert Test Cases:
  inv = Inventory()

```
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
```

Expected Output #4: • Fully functional class passing all assertions.

Output Screenshot:



Explanation: This simulates a mini store system where you can add, remove, and check stock for items.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

• Requirements:

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5: • Function passes all AI-generated assertions and handles edge cases.

Output Screenshot:

Explanation: This ensures a date is valid in “MM/DD/YYYY” format and neatly converts it into “YYYY-MM-DD.